线性分类模型

数据科学与大数据技术专业

第 3 周

1 分类问题与线性模型

1. **核心任务特性**
   - 分类任务需要将输入映射到离散标签空间, 通常需要非线性映射.
   - 应用场景广泛, 如图像识别、文本分类、情感分析等.
2. **线性分类模型构建**
   - **判别函数设计**:
     - 二分类: 通过单一决策边界将特征空间划分为两类.
     - 多分类: 可采用"一对其余"、"一对一"或"argmax"策略实现多类别判别.
       * **一对多 (One-vs-Rest, OvR)**: 为每个类别分别训练一个二分类器, 将当前类别视为正类, 其余所有类别合并视为负类. 预测时, 计算所有二分类器对输入样本的得分或概率, 选择输出最高的类别作为最终结果.
       * **一对一 (One-vs-One, OvO)**: 对任意两个类别分别训练一个二分类器, 因而总共需要训练 $\frac{C(C-1)}{2}$ 个分类器. 预测时, 让所有分类器分别进行两两判别并投票, 最终票数最高的类别作为预测结果.
       * **Argmax 策略**: 为每个类别构建一个判别函数, 给定特征后, 计算所有类别的得分或概率, 将得分或概率最高 (即 $\arg\max$) 的类别作为预测结果.
3. **分类评价指标**
   - **基础定义准备**: 给定测试集 $\{(\boldsymbol{x}^{(i)}, y^{(i)})\}_{i=1}^{N}$, 假设标签 $y^{(i)}$ 属于 C 个类别 (1 到 C). 使用学习好的模型 $f(\boldsymbol{x};\boldsymbol{\theta})$ 进行预测, 得到的预测结果记为 $\hat{y}^{(i)}$.
   - **整体性能指标**:
     - **准确率 (Accuracy)**: 表示所有预测正确的样本占总样本的比例:

$$\text{Accuracy} = \frac{1}{N}\sum_{n=1}^{N}\mathbb{I}(y^{(n)} = \hat{y}^{(n)})$$

       其中 $\mathbb{I}(\cdot)$ 为示性函数 (条件成立时为 1, 否则为 0). 准确率反映了模型在整个测试集上的综合正确表现.
     - **错误率 (Error Rate)**: 即分类错误的样本比例:

$$\text{Error Rate} = \frac{1}{N}\sum_{n=1}^{N}\mathbb{I}(y^{(n)} \neq \hat{y}^{(n)}) = 1 - \text{Accuracy}$$

- **类别级性能指标与混淆矩阵**: 为细致评估模型在特定类别 (尤其是目标正类, 记为 P, 其他为 N) 上的表现, 引入如下四个具体概念, 它们构成了一个二分类的**混淆矩阵**:

	预测为正类 ($\hat{y}=1$)	预测为负类 ($\hat{y}=0$)
真实为正类 ($y=1$)	真正例 (True Positive, TP)	假负例 (False Negative, FN)
真实为负类 ($y=0$)	假正例 (False Positive, FP)	真负例 (True Negative, TN)

  基于混淆矩阵, 可定义以下关键评价比率:

  - **精确率 (Precision)**: 定义为 $\frac{TP}{TP+FP}$. 它表示在所有被模型预测为" 正类" 的样本中, 真实确实为" 正类" 的比例. 精确率关注的是" 我找到的都是对的", 不在乎是否遗漏了部分正例. 例如在垃圾邮件过滤中, 如果邮箱里有 100 封垃圾邮件, 但模型只识别出了 50 封, 且这 50 封全部正确 (没有误伤正常邮件), 则精确率达到 100%, 但仍有 50 封垃圾邮件漏网之鱼. 当" 误报"(False Positives) 代价极高时 (如垃圾邮件拦截, 绝对不能把正常邮件判定为垃圾邮件而被错删), 应重点关注此项.
  - **召回率 (Recall)**: 定义为 $\frac{TP}{TP+FN}$. 它表示在所有" 真实正类" 样本中, 被模型成功找出来的比例. 召回率关注的是" 我是否找到了全部的正例", 不在乎预测结果中有多少误判. 例如在癌症筛查中, 如果有 100 位患者, 模型全部检测出来了 (100 个真正例), 但同时也错误地将 50 名健康人判定为患者 (50 个假正例), 则召回率仍是 100%, 尽管伴随大量误报. 当" 漏报"(False Negatives) 代价极高时 (如医疗诊断、安全检测等场景中, 绝不能漏掉任何潜在风险), 应重点关注此项.
  - **F_1 值 (F1 Score)**: 它是精确率和召回率的调和平均, 公式为:

$$F_1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2TP}{2TP + FP + FN}$$

    它能在保证较高精确率的同时不至于召回率太低, 是一种均衡两者表现的综合性局部指标.
  - **F_β 值 (F-Measure)**: 当业务场景对精确率和召回率的偏好不同时, 可以推广得到 F_β 值, 其公式为:

$$F_\beta = \frac{(1+\beta^2) \times \text{Precision} \times \text{Recall}}{\beta^2 \times \text{Precision} + \text{Recall}}$$

    其中, 参数 $\beta > 0$ 用于调节度量偏重:
    * 当 $\beta = 1$ 时, 退化为 F_1 值, 两者地位同等重要.
    * 当 $\beta > 1$ 时 (如 F_2), 意味着分母中召回率的权重被放大, 模型度量更**偏重召回率 (Recall)**, 适用于" 漏报" 代价远大于" 误报" 的情况.
    * 当 $0 < \beta < 1$ 时 (如 $F_{0.5}$), 则赋予精确率更大的权重, 模型度量更**偏重精确率 (Precision)**, 适用于" 误报" 代价极高的情况.

4. **典型线性分类器**

- **Logistic 回归**: 用于二分类概率建模.
- **Softmax 回归**: Logistic 回归的多分类扩展.
- **感知器**: 基于错误驱动的在线学习算法.
- **支持向量机 (SVM)**: 通过最大化间隔实现分类.

问题思考:

- 线性模型如何实现多分类?
- 感知器与 Logistic 回归的主要区别是什么?

2 信息论与损失函数

1. **交叉熵损失的引入：极大似然估计视角**

   - **概率输出建模**: 分类模型的输出可视作给定特征 $\boldsymbol{x}$ 下类别的条件概率分布 $P(y \mid \boldsymbol{x}; \boldsymbol{w})$, 而非直接输出确定的类别标签.
   - **极大似然估计 (MLE)**: 基于给定数据集 $\{(\boldsymbol{x}^{(i)}, y^{(i)})\}_{i=1}^n$, 模型参数 $\boldsymbol{w}$ 的学习目标转化为最大化观测数据的似然函数:
   $$\max_{\boldsymbol{w}} \prod_{i=1}^n P(y^{(i)} \mid \boldsymbol{x}^{(i)}; \boldsymbol{w})$$
   这等价于最大化对数似然 (Log-Likelihood):
   $$\max_{\boldsymbol{w}} \sum_{i=1}^n \log P(y^{(i)} \mid \boldsymbol{x}^{(i)}; \boldsymbol{w})$$
   即通过极大似然估计使模型给出的概率分布最大程度地拟合经验数据分布, 将问题从“计算误差距离”自然转化为“概率拟合”.

2. **二分类示例与 KL 散度的导出**

   - **交叉熵损失的形式导出与解释**:
     - **变量定义**: 在此场景中, $\boldsymbol{x}$ 代表给定的输入特征向量 (模型观测到的数据本身), 而 $y \in \{0, 1\}$ 代表客观存在的真实类别标签.
     - **预测概率** $p(\boldsymbol{x}) = P(y = 1 \mid \boldsymbol{x}; \boldsymbol{w})$: 指模型在给定参数前提下, 评估样本 $\boldsymbol{x}$ 属于正类 (通常指 $y = 1$) 的预估概率 (或置信度), 这是模型实际输出的具体数值.
     - **条件概率** $P(y \mid \boldsymbol{x}; \boldsymbol{w})$: 指标签 y 取任何真实值时对应的模型预测概率. 通过巧妙的指数构造 $P(y \mid \boldsymbol{x}; \boldsymbol{w}) = p(\boldsymbol{x})^y (1 - p(\boldsymbol{x}))^{1-y}$, 将 $y = 1$ 和 $y = 0$ 这两种情况统一融合为一个紧凑的数学解析表达式, 便于后续做对数与求导.
     - **目标函数的构建**: 给定独立同分布 (i.i.d.) 的样本集 $\{(\boldsymbol{x}^{(i)}, y^{(i)})\}_{i=1}^n$, 整个数据集的联合条件概率 (即似然函数) 为 $\prod_{i=1}^n P(y^{(i)} \mid \boldsymbol{x}^{(i)}; \boldsymbol{w})$. 对其取负对数 (Negative Log-Likelihood), 即将连乘转化为连加, 即可导出模型训练时需要最小化的目标函数:
     $$\mathcal{L}(\boldsymbol{w}) = -\sum_{i=1}^n \log P(y^{(i)} \mid \boldsymbol{x}^{(i)}; \boldsymbol{w}) = -\sum_{i=1}^n \left[y^{(i)} \log p(\boldsymbol{x}^{(i)}) + (1 - y^{(i)}) \log(1 - p(\boldsymbol{x}^{(i)})) \right]$$
     - **为什么它是“交叉熵”损失**: 在信息论定义中, 用分布 Q 去编码真实分布 P 所产生的交叉熵为 $H(P, Q) = -\sum P(c) \log Q(c)$. 在上述二分类中, 对于给定样本, 客观存在的真实概率分布为绝对笃定的结果 $[y, 1 - y]$(即如果标签真实为 1, 则属于标签 1 的概率为 1, 标签 0 的概率为 0), 而模型给出的预估概率分布为 $[p(\boldsymbol{x}), 1 - p(\boldsymbol{x})]$. 将两者代入公式, 单样本的交叉熵严格等于 $-[y \log p(\boldsymbol{x}) + (1 - y) \log(1 - p(\boldsymbol{x}))]$, 证明了负对数似然目标在数学本质上即是交叉熵.

- **信息论视角的解释**: 从信息论角度看, 交叉熵 $H(p,q) = -\mathbb{E}_p[\log q]$ 衡量了使用真实样本分布 p 评价模型预测分布 q 时的信息量差异.
- **极大似然与 KL 散度的内在等价性**: 相对熵 (KL 散度) 度量了真实分布与模型分布之间的距离. 由于 $\mathrm{KL}(p\|q) = H(p,q) - H(p)$, 且真实分布的熵 $H(p)$ 独立于模型参数, 因此最小化预测分布与真实分布间的 KL 散度本质上等同于最小化交叉熵损失, 这亦是极大似然估计在信息论下的等价体现.

3. **交叉熵与 KL 散度的严格数学定义**

- **信息熵 (Entropy)**: 对于离散随机变量, 设其真实概率分布为 $p(x)$, 信息熵衡量了该分布自带的不确定性, 定义为:
$$H(p) = -\sum_x p(x)\log p(x)$$
- **交叉熵 (Cross-Entropy)**: 当使用预测分布 $q(x)$ 去近似或编码真实分布 $p(x)$ 时, 所需付出的平均信息量, 定义为:
$$H(p,q) = -\sum_x p(x)\log q(x)$$
- **KL 散度 (Kullback-Leibler Divergence)**: 又称相对熵, 用于量化分布 q 偏离真实分布 p 所产生的"额外信息损失"(注: KL 散度不满足对称性, 不是严格意义上的距离度量), 定义为:
$$\mathrm{KL}(p,q) = \sum_x p(x)\log\frac{p(x)}{q(x)}$$
将对数项拆开, 能够清晰地揭示其与信息熵、交叉熵之间的加法等式关系:
$$\mathrm{KL}(p,q) = \sum_x p(x)\log p(x) - \sum_x p(x)\log q(x) = -H(p) + H(p,q)$$
即: 交叉熵 = 真实分布的熵 + KL 散度.
- **KL 散度非负性**: $\mathrm{KL}(p,q)$ 总是非负的, 事实上, 由对数函数的凹性可知
$$\log t \le t-1, \qquad \forall\, t>0.$$
对任意满足 $p(x)>0$ 的 x, 令
$$t = \frac{q(x)}{p(x)},$$
则有
$$\log\frac{q(x)}{p(x)} \le \frac{q(x)}{p(x)} - 1.$$
两边乘以 $p(x)$ 并在定义域上积分, 得到
$$\sum_x p(x)\log\frac{q(x)}{p(x)}\,dx \le \sum_x \big(q(x)-p(x)\big)\,dx = 0.$$
因而
$$\mathrm{KL}(p,q) = \sum_x p(x)\log\frac{p(x)}{q(x)}\,dx \ge 0.$$
等号成立当且仅当 $p(x)=q(x)$ 几乎处处成立.

问题思考:

- 为什么交叉熵损失适合分类问题?
- KL 散度与交叉熵的关系是什么?

3 Logistic 回归 (二分类)

1. **模型结构与预测概率**

   - Logistic 回归利用 Logistic(或 Sigmoid) 函数将线性映射输出转化为概率得分.
   - 给定输入特征 $\boldsymbol{x}$ 和模型参数 $\boldsymbol{w}$, 样本标签 y 等于 1 的条件概率计算公式为:

   $$p(y=1 \mid \boldsymbol{x}; \boldsymbol{w}) = \hat{y} = \frac{1}{1+\exp(-\boldsymbol{w}^\top \boldsymbol{x})}.$$

   - 定义 Logistic 函数 $\sigma(z) = \frac{1}{1+\exp(-z)}$, 则上述公式可简洁表示为:

   $$\hat{y} = \sigma(\boldsymbol{w}^\top \boldsymbol{x}).$$

2. **损失函数与经验风险**

   - 模型采用交叉熵作为损失函数.
   - 在包含 N 个样本的训练集上, 经验风险函数 (即平均交叉熵损失) 表示为:

   $$\mathcal{R}(\boldsymbol{w}) = -\frac{1}{N}\sum_{n=1}^{N}\left(y^{(n)}\log\hat{y}^{(n)} + \left(1-y^{(n)}\right)\log\left(1-\hat{y}^{(n)}\right)\right).$$

3. **模型优化与梯度计算**

   - 为了最小化经验风险, 通常采用梯度下降法 (Gradient Descent).
   - 风险函数关于参数 $\boldsymbol{w}$ 的梯度计算过程极为简洁: 由 Logistic 函数的导数公式:

   $$\sigma'(x) = \sigma(1-\sigma).$$

   可得

   $$\begin{aligned}
   \frac{\partial \mathcal{R}(\boldsymbol{w})}{\partial \boldsymbol{w}} &= -\frac{1}{N}\sum_{n=1}^{N}\left(y^{(n)}\frac{\hat{y}^{(n)}(1-\hat{y}^{(n)})}{\hat{y}^{(n)}}\boldsymbol{x}^{(n)} - \left(1-y^{(n)}\right)\frac{\hat{y}^{(n)}(1-\hat{y}^{(n)})}{1-\hat{y}^{(n)}}\boldsymbol{x}^{(n)}\right) \\
   &= -\frac{1}{N}\sum_{n=1}^{N}\left(y^{(n)}\left(1-\hat{y}^{(n)}\right)\boldsymbol{x}^{(n)} - \left(1-y^{(n)}\right)\hat{y}^{(n)}\boldsymbol{x}^{(n)}\right) \\
   &= \frac{1}{N}\sum_{n=1}^{N}\left(\hat{y}^{(n)} - y^{(n)}\right)\boldsymbol{x}^{(n)}.
   \end{aligned}$$

4. **参数迭代与算法停止策略**

- 根据求得的梯度, 模型参数的迭代更新公式为 (其中 α 为学习率):

$$\boldsymbol{w}_{t+1} \leftarrow \boldsymbol{w}_t - \alpha \frac{1}{N} \sum_{n=1}^{N} \left(\hat{y}_{\boldsymbol{w}_t}^{(n)} - y^{(n)} \right) \boldsymbol{x}^{(n)}.$$

- **早停策略 (Early Stopping)**: 对于 Logistic 回归, 训练目标是拉开类别间的概率差异, 无需苛求交叉熵降至极小. 若优化过程中损失过度减小, 可能导致参数 $\boldsymbol{w}$ 的范数急剧膨胀 (特别是对于靠近决策边界的样本, 将其概率推向绝对的 1 会导致参数发散). 因此, 在优化过程中适时停止训练至关重要, 以确保模型的泛化能力与数值稳定性.

问题思考:

- 为什么 Logistic 回归输出概率?
- 交叉熵损失如何影响模型训练?

4 Softmax 回归 (多分类)

1. **模型推广与分类策略**

   - Softmax 回归是 Logistic 回归在多分类问题上的自然推广.
   - 尽管模型本身依然是线性模型, 但配合 $\arg\max$ 策略, 能够将特征的决策平面划分为完全覆盖全空间的多个区域, 从而消除了不可判别区域.

2. **模型结构与概率映射**

   - 与二分类问题转为条件概率建模类似, 在多分类中, 核心挑战是如何将原本处于实数域 $\mathbb{R}$ 的相对得分 (C 个类别对应的输出) 映射为代表 C 个类别概率且总和为 1 的分布.
   - 模型通过引入 Softmax 函数进行概率归一化. 若得分为向量中某个元素 x_c, 其转化的预测概率计算公式为:

$$\mathrm{softmax}(x_c) = \frac{\exp(x_c)}{\sum_{i=1}^{C} \exp(x_i)}.$$

3. **损失函数与模型优化**

   - **损失函数**: 模型采用多元交叉熵损失 (Categorical Cross-Entropy), 以此为目标函数最大化真实正确类别的预测概率. 若训练集包含 N 个样本, 第 n 个样本的真实标签采用 one-hot 向量 $\boldsymbol{y}^{(n)} = (y_1^{(n)}, \ldots, y_C^{(n)})$, 模型预测概率为 $\hat{\boldsymbol{y}}^{(n)} = (\hat{y}_1^{(n)}, \ldots, \hat{y}_C^{(n)})$, 则经验风险函数可写为:

$$\mathcal{R} = -\frac{1}{N} \sum_{n=1}^{N} (\boldsymbol{y}^{(n)})^\top \log \hat{\boldsymbol{y}}^{(n)}.$$

   - **优化策略**: 依据损失函数通过梯度下降法等优化算法不断迭代更新模型参数, 有效地计算并拉开各类别之间相对得分的差距.

问题思考:

- Softmax 回归与 Logistic 回归的联系与区别?
- 多分类问题中, Softmax 函数的作用是什么?

4.1 补充: 为什么是 Softmax 函数

设 $x = (x_1, \ldots, x_C) \in \mathbb{R}^C$ 是给定的类别得分向量. 我们的目标是构造一个概率分布 $p = (p_1, \ldots, p_C)$, 满足

$$p_i > 0, \quad \sum_{i=1}^{C} p_i = 1,$$

并且这个分布能够以一种有理论依据的方式反映得分向量 x 中包含的信息.

下面从几个角度说明为什么会得到 Softmax.

最大熵视角 更具体地说, 我们希望这个概率分布所对应的平均得分, 能够反映样本集所给出的平均得分信息. 因此, 一个自然的要求是它满足如下期望约束:

$$\sum_{i=1}^{C} p_i x_i = \mu,$$

其中 μ 表示给定数据下观测到的经验平均得分, 或者更一般地说, 表示我们希望模型匹配的某个一阶统计量.

在满足这些约束的前提下, 我们还希望这个分布尽可能不额外引入新的信息, 也就是尽可能"无偏". 按照信息论中的观点, 这等价于在约束条件下最大化熵. 因此, 我们把求解概率分布 p 的问题转化为如下最大熵问题:

$$\max_p \left\{ -\sum_{i=1}^{C} p_i \log p_i \right\} \quad \text{subject to} \quad \sum_{i=1}^{C} p_i = 1, \quad \sum_{i=1}^{C} p_i x_i = \mu,$$

熵

$$H(p) = -\sum_{i=1}^{C} p_i \log p_i$$

衡量的是一个概率分布的不确定性程度. 分布越平均、越分散, 熵就越大; 分布越集中, 熵就越小. 如果只要求 $p_i > 0$ 且 $\sum_{i=1}^{C} p_i = 1$, 那么最大熵解必然是均匀分布 $p_i = 1/C$. 这虽然最"无偏", 却完全没有利用得分向量 x 的信息.

因此, 必须加入一条把概率分布与得分联系起来的约束. 选择

$$\sum_{i=1}^{C} p_i x_i = \mu$$

的好处在于: 它只要求分布在"平均得分"这一层面匹配数据中的已知信息, 而不强行规定更多细节. 换句话说, 这条约束负责把"高分应当对应更高偏好"这一信息注入模型, 而最大熵原则负责保证除了这些已知信息之外, 不再额外引入人为偏置.

引入拉格朗日乘子 $\alpha \in \mathbb{R}$ 和 $\lambda \in \mathbb{R}$, 可写出拉格朗日函数:

$$\mathcal{L}(p, \alpha, \lambda) = -\sum_{i=1}^{C} p_i \log p_i + \alpha \left(\sum_{i=1}^{C} p_i - 1 \right) + \lambda \left(\sum_{i=1}^{C} p_i x_i - \mu \right).$$

对每个 p_i 求偏导并令其为零, 有

$$\frac{\partial \mathcal{L}}{\partial p_i} = -(1 + \log p_i) + \alpha + \lambda x_i = 0,$$

从而得到

$$\log p_i = \alpha - 1 + \lambda x_i.$$

两边取指数可得

$$p_i = \exp(\alpha - 1)\ \exp(\lambda x_i).$$

再利用归一化条件 $\sum_{i=1}^{C} p_i = 1$, 就得到

$$p_i = \frac{\exp(\lambda x_i)}{\sum_{j=1}^{C} \exp(\lambda x_j)}.$$

这就是指数族分布在该约束条件下的标准形式. 当 $\lambda = 1$ 时, 上式恰好退化为标准的 Softmax 形式:

$$p_i = \frac{\exp(x_i)}{\sum_{j=1}^{C} \exp(x_j)}.$$

因此, Softmax 不是一个随意选出来的归一化公式, 而是"在固定期望得分约束下熵最大的分布"所自然导出的结果.

平移不变性 Softmax 函数还有一个重要性质, 即对平移不敏感. 对任意常数 $c \in \mathbb{R}$, 令 $\tilde{x}_i = x_i + c$, 则有

$$\frac{\exp(\tilde{x}_i)}{\sum_{j=1}^{C} \exp(\tilde{x}_j)} = \frac{\exp(x_i + c)}{\sum_{j=1}^{C} \exp(x_j + c)} = \frac{e^c \exp(x_i)}{e^c \sum_{j=1}^{C} \exp(x_j)} = \frac{\exp(x_i)}{\sum_{j=1}^{C} \exp(x_j)}.$$

因此, 最终得到的概率分布不会发生变化:

$$\mathrm{softmax}(x + c\mathbf{1}) = \mathrm{softmax}(x).$$

这表明 Softmax 只依赖于各类别得分之间的相对差异, 而不依赖于它们的绝对大小. 这一性质在统计学习和机器学习中非常重要, 也解释了为什么实际计算中常常先减去最大值来提高数值稳定性.

物理视角: 玻尔兹曼分布 从物理的角度看, Softmax 的形式与统计物理中的玻尔兹曼分布完全一致. 在热平衡状态下, 系统处于能量状态 E_i 的概率为

$$P(E_i) = \frac{\exp\left(-\frac{E_i}{kT}\right)}{\sum_j \exp\left(-\frac{E_j}{kT}\right)}.$$

如果把类别得分看成"负能量"或把温度参数吸收到得分缩放中, 就可以写成与 Softmax 完全相同的形式. 这说明 Softmax 也可以理解为一种"能量越低, 概率越大"或等价地"得分越高, 概率越大"的分布规律. 现代基于能量的模型 (Energy-based Models) 正是沿着这条思路展开建模的.

统计视角: 从 Logistic 到 Softmax 从统计学习的角度看, Softmax 也可以理解为二分类 Logistic 回归向多分类情形的自然推广. 在二分类中, 模型通过 Sigmoid 函数把一个实数得分映射为概率; 到了多分类情形, 我们需要把 C 个类别的得分同时映射为一个和为 1 的概率分布. 最自然的做法, 就是对每个类别的线性得分取指数, 再统一归一化, 从而得到 Softmax 形式. 因此, Softmax 既可以从最大熵推出, 也可以从 Logistic 回归的多分类推广得到.

交叉熵视角 Softmax 之所以在多分类中尤其常用, 还因为它和交叉熵损失配合后, 梯度形式会变得非常简洁. 设模型对 C 个类别输出的原始得分为

$$z = (z_1, \ldots, z_C),$$

经过 Softmax 后得到预测概率

$$\hat{y}_k = \frac{\exp(z_k)}{\sum_{j=1}^{C} \exp(z_j)}, \qquad k = 1, \ldots, C.$$

若真实标签采用 one-hot 形式记为

$$y = (y_1, \ldots, y_C), \qquad y_k \in \{0, 1\}, \qquad \sum_{k=1}^{C} y_k = 1,$$

则多分类交叉熵损失可写为

$$\mathcal{L} = -\sum_{k=1}^{C} y_k \log \hat{y}_k.$$

由于

$$\log \hat{y}_k = z_k - \log \sum_{j=1}^{C} \exp(z_j),$$

所以损失函数也可以改写为

$$\mathcal{L} = -\sum_{k=1}^{C} y_k z_k + \log \sum_{j=1}^{C} \exp(z_j).$$

对某个分量 z_m 求偏导, 有

$$\frac{\partial \mathcal{L}}{\partial z_m} = -y_m + \frac{\exp(z_m)}{\sum_{j=1}^{C} \exp(z_j)} = \hat{y}_m - y_m.$$

因此, 当我们**最小化交叉熵损失**时, 关于 Softmax 输入 z_m 的梯度正好就是

$$\frac{\partial \mathcal{L}}{\partial z_m} = \hat{y}_m - y_m.$$

这说明梯度仅仅等于"预测概率减去真实概率", 形式非常简单. 也正因为如此, Softmax 与交叉熵的组合在实际训练中既便于推导, 也便于实现.

历史补充 Softmax 这一名称通常追溯到 John S. Bridle 在 1989 年关于分类网络输出概率解释的工作 [1]. 在该工作中, 他把这种归一化指数映射明确用于多分类神经网络输出层, 从而使其成为今天机器学习中最常见的概率映射之一.

5 感知器 (Perceptron)

1. **模型背景与定义**

   - 感知器提出于 1958 年, 是早期机器学习领域的经典线性分类模型.

- 它采用线性判别函数, 并通过符号函数 (sign) 进行决策输出, 模型的数学表达为:

$$\hat{y} = \text{sign}(\boldsymbol{w}^\top \boldsymbol{x}).$$

2. **学习目标**

- 给定训练集 $\{(\boldsymbol{x}^{(n)}, y^{(n)})\}_{n=1}^N$, 感知器的目标是寻找最优权重 $\boldsymbol{w}^*$, 使得所有样本被正确分类. 这意味着对于全体样本均需满足:

$$y^{(n)}(\boldsymbol{w}^\top \boldsymbol{x}^{(n)}) > 0.$$

3. **错误驱动的学习与优化算法**

- **优化策略**: 感知器是一种基于错误驱动的在线学习算法.
- **执行过程**:
  (a) 初始化权重向量 $\boldsymbol{w} = \boldsymbol{0}$.
  (b) 当遇到分类错误的样本 $(\boldsymbol{x}, y)$ 时 (即满足约束条件 $y(\boldsymbol{w}^\top \boldsymbol{x}) \leq 0$ 的样本), 利用其信息对权重进行更新:

$$\boldsymbol{w}_{t+1} = \boldsymbol{w}_t + y\boldsymbol{x}.$$

  (c) 这一过程持续进行, 对于正确分类的样本则不更新模型参数, 直至没有错分样本为止.
- **损失函数形式**: 这种类似随机梯度的优化更新过程意味着, 感知器实际在最小化如下形式的 hinge 损失函数:

$$L = \max(0, -y(\boldsymbol{w}^\top \boldsymbol{x})).$$

4. **有效性与算法收敛性分析**

- **理论解释**: 假设在 t 时刻模型将某样本分错, 发生了更新 $\boldsymbol{w}_{t+1} = \boldsymbol{w}_t + y\boldsymbol{x}$. 在下次评估该样本点时:

$$y(\boldsymbol{w}_{t+1}^\top \boldsymbol{x}) = y(\boldsymbol{w}_t + y\boldsymbol{x})^\top \boldsymbol{x} = y(\boldsymbol{w}_t^\top \boldsymbol{x}) + y^2(\boldsymbol{x}^\top \boldsymbol{x}).$$

  由于 $y^2(\boldsymbol{x}^\top \boldsymbol{x}) > 0$, 因此模型关于该错误样本的判别得分朝着"正确方向 (变大)"进行了修正, 通过这种循环纠错最终实现学习.
- **线性可分前提**: 感知器算法理论上有限步终止且必然能够收敛, 其绝对前提是数据必须是"**线性可分**"的 (即存在一个超平面能将正负类样本完全分隔). 如果数据集非线性可分, 感知器的参数将会发散且永远无法收敛.

5. **与 Logistic 回归的对比**

- **触发条件差异**: 感知器在碰到不犯错的样本即不更新参数. 相反地, Logistic 回归若通过随机梯度下降更新, 利用的是错误概率"残差"; 只要预测概率并非绝对的 1 或 0, 即"分布存在偏差"或不够自信, 它便会持续微调. 它更新所用的是带有概率意义的残差量而不是真实的分类标签本体.

问题思考:

- 感知器算法为何只对分错样本更新?
- 感知器收敛性的前提条件是什么?

6 支持向量机 (SVM)

1. **最大间隔与模型鲁棒性**

   - 虽然感知器等算法能够找到多个不同的线性决策边界将样本完全分开, 但这些边界中往往存在距离某些样本极近的超平面, 这导致模型对微小扰动缺乏抵抗力.
   - **支持向量机 (SVM) 的核心思想**: 在所有能正确分类数据的超平面中, 寻找一个"最大程度远离所有两类样本点"的决策边界, 即寻找实现分类"间隔最大化"的最优超平面, 从而提供极高的抗扰动强健性.

2. **解析几何表示与硬间隔**

   - 设决策平面的方程为 $\boldsymbol{w}^\top \boldsymbol{x} + b = 0$. 根据解析几何, 训练样本 $(\boldsymbol{x}, y)$ (其中 $y \in \{+1, -1\}$) 到该超平面的几何距离 γ 可表示为:

   $$\gamma = \frac{y(\boldsymbol{w}^\top \boldsymbol{x} + b)}{\|\boldsymbol{w}\|}.$$

   - SVM 的首要目标即是在满足正确分类的基础之上, 寻找最优的权重 $\boldsymbol{w}$ 和偏置 b 使得最小的几何距离 γ 最大化. 此情况即要求数据严格线性可分的"**硬间隔**"SVM.
   - **尺度归一化与等价优化问题**: 若直接将硬间隔 SVM 写为

   $$\begin{array}{ll} \max_{\boldsymbol{w},b,\gamma} & \gamma \\ \text{s.t.} & \frac{y^{(n)}\left(\boldsymbol{w}^\top \boldsymbol{x}^{(n)} + b\right)}{\|\boldsymbol{w}\|} \geq \gamma, \quad \forall n \in \{1, \ldots, N\}, \end{array}$$

   则这个形式存在**尺度不唯一**的问题: 对任意 $k > 0$, 将 $(\boldsymbol{w}, b)$ 同时缩放为 $(k\boldsymbol{w}, kb)$, 并不会改变分离超平面, 也不会改变样本点到超平面的几何距离. 因而该问题的参数自由度过高, 不便于直接求解. 为了消除这种缩放不确定性, 通常加入归一化约束 $\|\boldsymbol{w}\| \cdot \gamma = 1$, 等价地把问题改写为

   $$\begin{array}{ll} \max_{\boldsymbol{w},b} & \frac{1}{\|\boldsymbol{w}\|} \\ \text{s.t.} & y^{(n)}\left(\boldsymbol{w}^\top \boldsymbol{x}^{(n)} + b\right) \geq 1, \quad \forall n \in \{1, \ldots, N\}. \end{array}$$

   进一步地, 由于最大化 $\frac{1}{\|\boldsymbol{w}\|}$ 等价于最小化 $\frac{1}{2}\|\boldsymbol{w}\|^2$, 硬间隔 SVM 最终可转化为标准凸优化问题:

   $$\begin{array}{ll} \min_{\boldsymbol{w},b} & \frac{1}{2}\|\boldsymbol{w}\|^2 \\ \text{s.t.} & y^{(n)}\left(\boldsymbol{w}^\top \boldsymbol{x}^{(n)} + b\right) \geq 1, \quad \forall n \in \{1, \ldots, N\}. \end{array}$$

3. **容错与软间隔**

   - **松弛变量引入**: 现实数据存在噪声或者本身非线性可分. 如果强制要求无错划分, 会找不到最优决策平面或者引发严重过拟合.
   - **软间隔优化目标与参数 C**: 为解决此问题, 我们引入松弛变量 (Slack Variables) $\xi_n \geq 0$, 容许部分样本落入间隔内部甚至被分错, 同时对这种违反间隔约束的程度进行惩罚. 此时软

间隔 SVM 可写为如下优化问题:

$$\begin{array}{ll} \min_{\boldsymbol{w},b,\boldsymbol{\xi}} & \frac{1}{2}\|\boldsymbol{w}\|^2 + C\sum_{n=1}^{N}\xi_n \\ \text{s.t.} & y^{(n)}\left(\boldsymbol{w}^\top\boldsymbol{x}^{(n)}+b\right) \geq 1-\xi_n, \quad \forall n\in\{1,\ldots,N\}, \\ & \xi_n \geq 0, \quad \forall n\in\{1,\ldots,N\}. \end{array}$$

这说明所谓"**软间隔**", 本质上就是对硬间隔 SVM 中的约束

$$y^{(n)}\left(\boldsymbol{w}^\top\boldsymbol{x}^{(n)}+b\right) \geq 1$$

进行了放松: 现在允许 $y^{(n)}(\boldsymbol{w}^\top\boldsymbol{x}^{(n)}+b) < 1$, 但不能偏离得太多, 偏离的程度由 ξ_n 来刻画. 当 $0 < \xi_n < 1$ 时, 样本虽然仍被分对, 但已经落入间隔内部; 当 $\xi_n \geq 1$ 时, 样本就可能被错分. 常数 $C > 0$ 用于平衡"最大化间隔"和"惩罚违约"这两个目标: 当 C 非常大时, 优化会强烈压低 ξ_n, 从而要求样本尽量满足硬间隔约束, 模型会更趋向于线性可分的情形; 当 C 较小时, 模型则会容许更多样本违反间隔约束, 以换取更大的间隔和更强的鲁棒性.

- **与 Hinge 损失的等价形式**: 数学上, 上述带松弛变量的软间隔支持向量机, 可以等价地转化为最小化基于 Hinge 损失的"经验风险"再加上一项"正则化项". 具体而言, 即为求解如下目标函数的最小化参数 (权重 $\boldsymbol{w}$ 与偏置 b):

$$\min_{\boldsymbol{w},b}\sum_{n=1}^{N}\max\left(0, 1-y^{(n)}\left(\boldsymbol{w}^\top\boldsymbol{x}^{(n)}+b\right)\right) + \frac{1}{2C}\|\boldsymbol{w}\|^2.$$

  在这个表达式中, 前半部分的 Hinge 损失总和用于衡量样本分类的经验误差 (容忍少量错分), 后半部分的正则化项用于最大化间隔距离, 超参数 C 用于调节两者的平衡. 这种等价转化在保障分类能力的同时, 显著提升了模型的泛化性.

4. **利用损失函数对比解析 SVM(及其它模型)**

- 考虑损失函数时, 常定义 $yf(\boldsymbol{x})$ 为"分类裕度"(Margin), 其值可看作样本距离决策边界的带符号距离: 值越大证明分类越确信正确, 值越小 (特别是负值) 说明分错越离谱.
- **平方损失**:$\mathcal{L} = (1-yf(\boldsymbol{x}))^2$. 即使 $yf(\boldsymbol{x}) > 1$(预测极端正确), 依然会累积巨大的惩罚. 这是违背分类直觉的, 因此绝对不适合用于分类模型.
- **Logistic 回归损失**: $\mathcal{L} = \log(1+\exp(-yf(\boldsymbol{x})))$. 尽管随着 $yf(\boldsymbol{x})$ 增大, 损失呈指数级衰减, 但对绝对正确的预测依然保留了一个虽小但不为零的惩罚.
- **感知器损失**: $\mathcal{L} = \max(0, -yf(\boldsymbol{x}))$. 从分类直觉出发, 分对 (距离 > 0) 即无任何惩罚.
- **SVM Hinge 损失**: $\mathcal{L} = \max(0, 1-yf(\boldsymbol{x}))$. 软间隔 SVM 使用 Hinge 损失 $\max(0, 1-yf(\boldsymbol{x}))$. 它相当于将感知器的损失函数进行"平移", 强制要求正确侧必须有着越过"间隔 1"的距离才算真正安全. 对于只是刚刚分对但贴近决策边界的点, 依旧会受到惩罚, 进一步确立了其模型更高的健壮性.

5. **优化方法**

- 支持向量机在引入损失函数后, 原问题经常被转化为对偶凸二次规划 (Quadratic Programming) 模型进行精确解析, 常用 SMO(Sequential Minimal Optimization) 等序列最小优化算法求解.

问题思考:

- SVM 为何要最大化间隔?
- 软间隔 SVM 如何处理噪声数据?

7 模型对比总结

- 下表总结了各线性模型的激活函数、损失函数、优化方法及典型应用场景:

模型	激活函数	损失函数	优化方法	主要应用场景
线性回归	-	均方误差	最小二乘/梯度下降	回归任务
Logistic 回归	Sigmoid	二元交叉熵	梯度下降	二分类概率预测
Softmax 回归	Softmax	多元交叉熵	梯度下降	多分类概率预测
感知器	符号函数 (sgn)	错误驱动	随机梯度下降	线性可分分类
SVM	符号函数 (sgn)	Hinge 损失 + 正则化项	二次规划/SMO	最大间隔分类

问题思考:

- 不同模型的损失函数有何异同?
- 线性模型在实际应用中如何选择?

(注: 详细定义和更多的内容请参考课程教材第 3 章, 附录 E.)

参考文献

[1] John S. Bridle. Probabilistic interpretation of feedforward classification network outputs, with relationships to statistical pattern recognition. In *Neurocomputing: Algorithms, Architectures and Applications*, pages 227–236. Springer Berlin Heidelberg, Berlin, Heidelberg, 1990.